



A DISCUSSION PAPER

The Bottleneck Has Moved

What if every developer became a Product Owner?
A thought experiment for the agentic era.

An optional, experimental layer on top of SAFe, Scrum, LeSS or Kanban
Jesper Karlsson · Otrobonita AI Labs

IF YOU READ ONE PAGE, READ THIS ONE.

The Short Version

Development capacity is no longer the scarce resource. **Judgment and accountability are.**

Most software organizations are still built to ration developer time: backlogs, story-point estimation, sprint capacity planning. That machinery answers a question that is quietly ceasing to matter. When execution becomes cheap and fast, the constraint moves — to the rate at which an organization can generate ideas worth building, and to who is answerable for what ships.

The hypothesis. Dissolve development teams into **one-human agentic teams**. Each developer becomes a Product Owner with full accountability for a value stream, supported by a cast of agents. Four teams of five become twenty accountable owners. Nobody changes desks; nobody is laid off. What changes is where accountability lives.

Why accountability is the crux. Agents can execute. They cannot be accountable. “The team was accountable” satisfies no regulator, auditor or customer once agents are making changes at speed and scale. One human, clearly named, per value stream is the only configuration that survives contact with governance.

It is a plugin, not a replacement. It sits on top of SAFe, Scrum, LeSS or Kanban and changes who is accountable underneath the existing cadence — not the cadence itself.

The number, honestly. If each agentic team delivered half of what a five-person team delivered, capacity would grow roughly 10–12× at flat salary cost. Field experience suggests 2–5× on well-bounded work is more realistic, with heavier coordination costs on integrated systems. *The multiple is uncertain. The direction is not.*

Where it does not work. Wherever integration density, regulatory coupling or legacy entanglement is high — core banking, telecom network functions, safety-critical systems, deeply shared domain models. There the dominant cost was never writing code; it was coordinating change, and twenty parallel streams multiply that surface rather than reduce it. This belongs at the edges, not in the regulated core.

What it costs. Compute becomes a real operating expense alongside salary. Verification becomes a system the accountable human owns rather than work they personally do. Peer contact becomes voluntary, and therefore uneven — a problem management has to solve deliberately.

Read the open questions before you argue. Architectural consistency across twenty streams is unresolved here, and it is the precondition for everything above. So is auditing accountability without producing fear, and stopping verification from decaying into rubber-stamping. They are stated plainly in Section 8.

This is a hypothesis offered for argument — not a consulting offering, and not a recommended operating model.

0. What This Paper Is, and Isn't

This paper is offered as a point of view — a hypothesis to argue with, not a consulting offering and not a recommended operating model. The author would be the first to be surprised if any organization adopted it wholesale. Its purpose is narrower and, hopefully, more useful: to make the current arrangement feel less inevitable, and to give organizations that are starting to think about agentic team structures a concrete strawman to sharpen their own thinking against.

Everything that follows is deliberately specific. Vague provocations are easy to nod at and easier to dismiss; a specific one gives you something to disagree with productively. Where the paper sounds confident, read it as *precision*, not certainty. This is one possible response to the agentic era — not *the* response.

1. The Hypothesis in One Breath

Development capacity is no longer scarce; judgment and accountability are. Organizations still structured around capacity scarcity — backlogs, sprint planning, resource allocation — are optimizing for a constraint that no longer exists.

So here is the thought experiment: what if traditional development teams dissolved into one-human agentic teams, where each developer becomes a Product Owner with full accountability for a value stream? Not as a reorganization — as a **plugin**: a layer that sits on top of whichever framework an organization already runs — SAFe, Scrum, LeSS, or Kanban — and changes who is accountable underneath the existing cadence, not the cadence itself.

Let us be honest about the scale of this from the start: no one changes desks, no one is laid off, and reporting lines can remain — but a shift in accountability of this magnitude is a real change in power, status, and career paths. If an organization ever tried this, it would be a *manageable* change, not an invisible one.

1.5 Framework Compatibility at a Glance

Framework	Stays Untouched	What the Plugin Would Replace
SAFe	PI planning cadence, ARTs, portfolio-level strategic themes, existing governance boards	Team-level backlog; single PO-per-team becomes PO-per-agentic-team; Iteration Review becomes the Retrospective Portfolio Showcase
Scrum	Sprint cadence (optional — many teams would compress it), Definition of Done, the retrospective ritual itself	The backlog as a pre-planned queue; PO role expands from one team to one full value stream; Scrum Master duties absorbed by the Orchestrator agent
LeSS	Cross-team coordination principles, single Product Backlog <i>philosophy</i> (though its contents change)	Multi-team Sprint Planning; the "feature team" becomes a single-human agentic team
Kanban	The board itself, WIP limits as a concept, flow metrics	What flows through the board — agent-executed work items replace human ticket-pulling; the board becomes the Orchestrator's dashboard

Kanban is the weakest fit of the four — it has no PO concept to begin with, so accountable-owner roles would need to be introduced rather than remapped. Flagged openly rather than claiming frictionless universal fit.

2. The Spark: The Scarcity Inverted

The fastest organizations are not marginally faster than their peers — they compound. Ship, learn, ship again, in cycles measured in days rather than quarters, and the gap widens every cycle rather than staying fixed.

For the first time, the constraint on that loop is not delivery capacity. It's the rate at which management can generate ideas worth building. An organization that cannot produce demand as fast as it can produce solutions has an unfamiliar problem — and an unusual opportunity, because the alternative to idea scarcity is *choice*: the ability to fund only the best ideas, tested against real output, rather than the ideas that survived a planning cycle.

Put provocatively: *most organizations are organized for the wrong century of scarcity.* Backlogs, story-point estimation, and sprint capacity planning are all machinery built to ration a scarce resource — developer time. If that resource is no longer the bottleneck, the machinery built to manage it is now overhead. That "if" is worth sitting with, because it is quietly becoming a "since."

3. The Accountability Problem

Agents can execute. They cannot be accountable. Accountability is non-delegable — a fact regulators, auditors, and customers already assume even where organizations haven't caught up to it.

Today's accountability is diffuse by design: collective code ownership, shared sprint commitments, a team that succeeds or fails together. That diffusion was tolerable when the worst outcome was a missed sprint goal. It becomes untenable once agents are making changes at speed and scale, because "the team was accountable" is not an answer that satisfies anyone standing outside the team when something goes wrong.

The core of the hypothesis: the only stable configuration going forward is one human, clearly accountable, per agentic team. Every agentic team traces back to exactly one name.

If that's right, it is arguably *more* SAFe-compliant than the status quo, not less — accountability becomes traceable per individual rather than collectively vague, which is precisely what governance and audit functions already want and rarely get.

One caveat this paper takes seriously: a named human is a *necessary* condition for real accountability in the agentic era, not a *sufficient* one. Personal accountability without support structures produces risk-aversion and fear, not empowerment. Sections 5 and the Open Questions address what those support structures would need to include.

3.5 The Psychological Bureaucracy Trap

Before going further, rid yourself of one instinct: counting the lone human. It is the single most common misreading of this model — and one that infected early drafts of this very paper.

An agentic team is not "one person working alone with tools." It is a complete team: it attends the remaining ceremonies, informs other teams of ongoing work, flags duplicates, and coordinates across streams — the difference is that most of that coordination is carried by agents, whose time is cheap. Coordination is not removed by this model; it is lifted to the Product Owner layer and the showcase cycle (see Open Questions). A no-silo mentality that was always expensive to maintain with humans becomes nearly free with agents: cross-team awareness, duplicate detection, and dependency flagging can run continuously rather than waiting for the next sync meeting. Mistakes surface far earlier than in the current state, not later.

This matters most when reading the support roles in Section 5. The Portfolio Curator, Demand Router, and Governance Advisor are not three humans forming a new commit-

tee — they are agentic teams like every other, each with one accountable human and an agent cast. The moment you picture them as a human coordination layer, you have rebuilt the bureaucracy this thought experiment exists to retire. The plugin is, at its core, about setting responsibility thresholds — who is accountable for what — not about reducing headcount per team to one.

4. The Model: From 4 Teams to 20

Before: 4 teams $\times$ 5 developers = 20 producing units, 4 Product Owners, 4 backlogs.

After: 20 agentic teams, 20 Product Owners, dynamic intent queues instead of pre-planned backlogs.

The math — clearly labeled as hypothetical: the numbers below are an illustration of the *shape* of the opportunity, not a forecast. Real-world multipliers depend heavily on domain complexity, integration load, and how much of the work is genuinely parallelizable.

If each agentic team delivered 50–60% of what a traditional five-person team delivered, total delivery capacity would grow roughly 10–12 $\times$ at flat salary cost. Current field experience suggests more modest multiples — often 2–5 $\times$ on well-bounded work — and heavier coordination costs on deeply integrated systems. Even at the conservative end, the direction is interesting: significant capacity growth with zero net hires. Add the calendar-time effect: agentic teams don't sleep, so elapsed time to delivery compresses independently of the throughput multiple.

The honest version of this section's claim: *the multiple is uncertain; the direction is not.*

Where This Model Is Weakest

The multiple varies with domain, and in some domains it collapses. This model is weakest wherever **integration density, regulatory coupling, or legacy entanglement is high** — core banking platforms, telecom network functions, safety-critical and certified systems, and deeply shared domain models inside long-lived monoliths.

The reason is structural rather than technological. In these environments the dominant cost was never writing the code; it was coordinating the change — impact analysis across a shared schema, certification and audit trails, release trains synchronized across dependent systems. Twenty parallel streams do not reduce that coordination load. They multiply the surface that has to be coordinated, which is the opposite of what the capacity argument assumes.

That does not make the idea inapplicable in these organizations, but it does change where it would apply. The plugin fits best at the edges — new products, customer-facing surfaces, internal tooling, data and analytics work, greenfield services — and fits worst at the regulated, tightly-coupled core. A bank might reasonably run twenty agentic teams across digital channels and internal platforms while leaving the ledger under its existing structure. That would not be partial adoption failure; it would be correct application.

Stating this plainly costs nothing and buys credibility. A model claimed to work everywhere is a model no experienced architect will trust anywhere.

5. Inside an Agentic Team: Roles and Titles

Each agentic team has exactly one human. Beyond that, the cast below is illustrative, not prescriptive — what is fixed is the structure (one human, accountable; agents covering the functions the value stream needs), while the composition, naming, and *size* of the agent cast is each Product Owner's own call. Some POs would run five agents; some practitioners already run fifty today. The right number is not set by this paper — it is set by the human's comfortable span of accountability: how much agent output one person can genuinely stand behind. That makes the model self-regulating in a way no prescribed team size could be — the moment a PO can no longer honestly answer for what their agents ship, scaling down or strengthening their verification layer is their call, and their responsibility. Designing one's team is not a deviation from the model; it is the first act of ownership.

That span of accountability is bounded by two practical constraints, and both belong in the budget. The first is **human cognitive bandwidth** — how much output one person can genuinely review without rubber-stamping it. The second is **compute budget**: token consumption and model compute increasingly replace headcount as the primary operational expense, and many parallel multi-agent teams running long chains is a visible running cost, not a rounding error. Human oversight time is usually the binding constraint on quality and accountability; compute is the one that shows up on an invoice. Balancing execution speed against token burn is therefore part of product ownership, not invisible IT infrastructure overhead — and the two constraints together, rather than any rule in this paper, are what set the practical ceiling on team size.

Two functions are non-negotiable in some form, because the rest of the model depends on them: **verification** (the Reviewer and Specification QA functions below, however named) and an **adoption-ready record** (the Chronicler function) — without the first, accountability is nominal; without the second, the Retrospective Portfolio Showcase in Section 6 has nothing to evaluate.

An example cast:

- **The Product Owner** (human) — the only role with mandate and accountability
- **The Orchestrator** — distributes work among agents, manages flow, escalates to the human
- **The Specifier** — translates intent into buildable specifications
- **The Builders** — one or more implementation agents, possibly different models for different strengths
- **The Reviewer** — QC, test, and security; veto power before release
- **The Chronicler** — documentation, changelog, and the record the PO later presents at showcase
- **The Scout** — external signal: competitors, demand signals, emerging needs

A simple org diagram of this cast does a lot of work — a reader can *see* a team of seven, six of them software.

A note on agent maturity, honestly: as of this writing, agents remain imperfect on long chains of work — hallucination, weak cross-session memory, and subtle intent drift are real. The roles above describe a *direction of delegation*, not a claim that today's agents perform them flawlessly. The human-in-the-loop cost is real and must be budgeted, not assumed away. This is precisely why the verification-oriented roles matter as much as the generative ones.

Where Today's Requirements Analysts Go: Specification QA

The Specifier produces a first-draft translation of intent into a buildable spec — a fast, confident draft, but not automatically a correct one. This is exactly where agents are most prone to quietly hallucinating an interpretation nobody asked for. A Requirements Analyst's real craft was never writing user stories; it was catching ambiguity before it became expensive. That craft now lands one step earlier in the pipeline: checking the Specifier's output against actual business intent, before a single Builder spends compute on it.

This is distinct from the Reviewer role. The Reviewer catches bad *output*, after building. Specification QA catches a bad *spec*, before building starts — the shift-left argument in its purest form. Together, these two verification layers are also the primary answer to the cognitive-overload concern: the accountable human is not expected to personally re-verify 10× more output; they are expected to own a verification *system* and spot-check it, the way an engineering manager owns quality today without reading every line.

Where Today's Product Owners Go: Up, Not Out

Dissolving twenty backlogs into twenty accountable individuals would not eliminate the skills today's Product Owners carry — it would relocate them to where the new risk actually concentrates.

Portfolio Curator (or *Portfolio Steward*): runs the Retrospective Portfolio Showcase described in Section 6 — organizing the showcase cycle, helping leadership compare deliveries across teams, and keeping adoption decisions anchored to demonstrated impact and strategic fit rather than to whoever presents best. This is PO craft exercised one level up, across the whole portfolio instead of one backlog — arguably the most senior role in the entire system.

Demand Router (or *Intake Steward*): the switchboard for everything that used to land in a backlog — customer requests, compliance mandates, executive ideas. Not a queue teams work through, but a routing function that connects incoming signal to the PO best placed to decide whether to act on it. Prioritization stays with the twenty POs; this role ensures nothing worth hearing is lost in the noise of twenty parallel streams.

Coach and Governance Advisor: newly-minted POs would be absorbing accountability many have never carried before, on two fronts. One is *craft* — framing value, saying no to a stakeholder, presenting a delivery with confidence — where a former PO is a natural mentor. The other is *exposure* — a developer-turned-PO is now the accountable human for decisions an agent executed: contractual commitments, IP provenance in agent-generated work, data handling, liability if an agent's output causes downstream harm. That second front is not a coaching problem, it's a governance one, and argues for pairing every new PO with standing legal and compliance access rather than leaving them to discover the exposure on their own.

A candid caveat: these three functions are load-bearing. If Curation, Routing, and Governance underperform, the model degrades into twenty parallel priority fights. Any organization experimenting with this would need to staff the accountable humans in these roles with their best people, not their leftover capacity — and define their decision rights and success metrics explicitly before scaling past a pilot.

Two clarifications to prevent the most likely misreading (see Section 3.5): first, these are agentic teams like every other — one accountable human each, with an agent cast doing most of the continuous work. A Curator's agents compare deliveries and track strategy alignment around the clock; a Router's agents classify and route incoming signal in real time. Second, they are *enabling services*, not gatekeepers: pull-based functions the twenty POs draw on when needed, not approval layers work must pass through. The moment any of the three requires a standing meeting to proceed, it has drifted into exactly the bureaucracy this model exists to retire.

6. The Inverted Portfolio Gate: From Pre-Commitment to Adoption Decisions

This is the most radical part of the hypothesis, so it is sketched concretely — not because the mechanics are settled, but because a concrete sketch is easier to argue with than an abstract one.

Instead of pre-planned backlogs, needs are elaborated as they arise, and Product Owners present *working, adoption-ready deliveries* at a recurring **Retrospective Portfolio Showcase**.

Leadership and domain stakeholders no longer approve backlogs in advance. They evaluate working software and make **adoption decisions** — determining which proven deliveries are taken into production, integrated, scaled, and funded onward. Value is established after something exists, not before. Deliveries that receive no adoption decision may continue locally, but without additional organizational support.

Management's role sharpens rather than shrinks: set direction, decide on adoption, fund what proves itself. Employees elaborate and refine, with innovation space structural to the model rather than bolted on as a separate ritual.

What an Adoption Decision Might Allocate

"Adoption" is not an abstract endorsement, and it is not primarily about money. A decision could allocate one or more of:

- **Priority access to shared resources** — platform teams, data engineering, security review, compliance capacity
- **Rollout funding** — budget for production hardening, integration, and scaling
- **Official portfolio adoption** — formal status and visibility as a supported solution
- **Expanded agent and compute capacity** — access to more parallel agents or premium models (see Section 5: compute is now a real allocation lever, not overhead)

Read together: an adoption decision signals how much the organization is willing to invest in taking further something that already demonstrably works.

Two Tracks, Not One

	Exploratory Value Streams	Maintenance Covenant
Funding	Competitive — earned through adoption decisions	Guaranteed, non-competitive
When value is determined	After delivery, at showcase	In advance, collectively by the POs
Visibility	Showcase + adoption decision	Equal reporting visibility
Examples	New features, experiments, spikes	Security, refactoring, infrastructure, compliance

Figure 1 — Two funding tracks.

No leadership team will ever compete to fund a database migration, and the model must not pretend otherwise. Keep-the-lights-on work is owned collectively by the accountable POs whose value streams depend on it, funded by covenant, and reported with the same visibility as adopted deliveries — never left to win attention against a demo.

Contended Resources and Arbitration

When two or more deliveries compete for the same constrained resource — a shared data platform, a single security review slot, limited compliance capacity — one plausible mechanism: the conflict is resolved by the Portfolio Curator together with the affected Product Owners, with strategic themes as the deciding criterion. If no agreement is reached within a defined time window, the decision escalates to whoever owns those strategic themes. The goal is fast clarity, not consensus at any cost.

Cultural Variant: Competitive or Collegial

The inverted logic does not require internal rivalry. Two modes seem equally plausible, and the choice is cultural rather than structural:

Track A — Competitive Selection. Suited to high-rivalry, market-driven environments. Parallel agentic teams may pursue competing implementations of a strategic intent, and leadership funds the strongest demonstrated result.

Track B — Collegial Allocation. Designed for consensus-driven, Scandinavian, or deeply integrated enterprise cultures. The Product Owners operate as a peer council: reviewing deliveries together, harmonizing domain boundaries, and presenting a unified, modular portfolio to leadership for adoption decisions.

Both preserve the core inversion — value determined after delivery rather than predicted before it. Only the social mechanism differs.

The Anti-Pattern to Watch For

The most likely degeneration is the showcase becoming a contest in presentation skill. The Portfolio Curator's most important job is keeping evaluation anchored in demonstrated impact and strategic fit, not in how convincingly someone presents. **If presentation begins to outweigh result, the mechanism has failed** — and that is a signal to intervene, not a cost of doing business.

The Obvious Objection, Addressed Directly

Doesn't this create chaos? Direction remains centrally set; only *elaboration* is decentralized. But the harder version of the objection deserves acknowledgment: with twenty parallel streams, elaboration *can* drift into de facto strategy. The Portfolio Curator exists precisely to catch that drift.

More fundamentally: without working mechanisms for architectural alignment and shared contracts, this model rewards locally optimal but globally inconsistent solutions. **The Open Questions section is therefore not an appendix — it is a list of preconditions** any serious attempt would have to answer first.

7. What This Would Demand of Leadership

Conceding real costs builds more credibility than omitting them.

- Management becomes the bottleneck by design; that requires faster, braver adoption decisions than most governance boards are used to making.
- Not every developer wants to be a Product Owner. Some would need real support growing into ownership, and there should be an honorable path — see Section 5 — for those who don't. In a larger organization, that path must be real and resourced, not a face-saving label.
- Governance must shift from process compliance to outcome accountability, which is a harder thing to audit but a truer thing to measure.
- Personal accountability at this scale requires psychological safety by design: clear escalation rights, blameless failure review for good-faith decisions, and explicit limits on individual liability. Accountability that feels like personal legal jeopardy produces paralysis, not speed.

8. Open Questions

A thought experiment earns trust by naming what it hasn't answered. A scoping note first:

If you already have a great team setup — dynamic co-work, few earbuds, genuine fun — this idea might not be for you. It's aimed at organizations where that dynamic has already eroded, or never existed at scale, not at replacing one that's working.

The questions below are named rather than solved — deliberately. Organizations start from different enough places that a single prescribed fix would be more confident than honest, and several of these are exactly where the author hopes readers will push back hardest.

The Scarcity of Social Touchpoints The shift to one-human agentic teams does not remove colleagues, desks, or the coffee machine. The physical and social fabric stays intact. What changes is the composition of the daily working unit — and, gradually, the topics that dominate hallway and after-work conversation.

Nor do the coordination rituals disappear. Local, intra-team ceremonies change character: with one human in the working unit, stand-ups and detailed sprint planning become lighter and more agent-supported. But the underlying need for synchronization, dependency management, architectural alignment, and collective prioritization is *lifted*, not removed — it moves to the Product Owner layer, the guilds, and the recurring Retrospective Portfolio Showcase. What used to happen inside a team now happens between accountable owners, supported by agents maintaining continuous situational awareness at low cost.

What genuinely thins, if left unattended, is *ambient* human contact — the informal check-in, the shared sprint goal, the spontaneous pairing. It's worth noting the baseline here is often less rich than it appears: many developers already work heads-down, isolated by focus if not by structure. This model doesn't invent isolation. But it does make peer contact more voluntary, and therefore more unevenly distributed.

Management would need to treat the scarcity of natural human-to-human touchpoints as an explicit concern, and deliberately introduce contact points that fit the organization's existing culture. These need not be new formal ceremonies — this paper deliberately prescribes none. Agentic comparison and evaluation sessions, cross-stream pairing, informal PO gatherings, guild-style domain discussions, or simply protected time for informal exchange can all serve, provided they are real, recurring, and culturally natural rather than bolted on.

The goal is not to recreate the old team feeling. It is to ensure that accountability does not quietly turn into isolation.

Architectural Consistency — the Largest Open Question Twenty accountable teams generating code independently only works if there's a shared floor underneath them — common data models, service contracts, enough architectural governance that twenty solutions don't quietly become twenty incompatible ones. This paper doesn't prescribe that governance layer; it will look different in a bank than in a media company, and designing it well is genuinely hard. What the hypothesis does insist on: the accountable human in each agentic team is accountable for architectural *alignment*, not just feature delivery — and any organization experimenting with this would need to decide, deliberately, who owns the shared contracts before the first team ships. This is the single most important precondition for the capacity math in Section 4 holding at all — and the question where architects' pushback would teach the most.

Controlled Duplication: Exploration vs. Core Systems Cheap execution means duplication, and duplication is not uniformly good or bad. The line runs between exploration and operation, and it must be drawn sharply:

Core systems and continuous operation. Large integrated applications, shared domain logic, common data schemas, and API contracts are **never** subject to uncoordinated duplicate building. They remain under explicit collective ownership and continuous governance of the PO group or guild. Architectural consistency here is non-negotiable — this is precisely where two teams solving "the same thing differently" produces conflicting business logic and duplicated endpoints in production rather than useful variety.

Proof-of-concepts and exploratory spikes. Duplication is welcome, and cheap. Two agentic teams independently attacking a novel feature in an afternoon is a low-cost discovery mechanism, not a planning failure. Don't prevent it in advance; let it happen, then pick the cherries.

A useful precedent for the exploratory side: Microsoft's internal app catalogue holds well over a million internally built applications. The overwhelming majority never reach a wider audience — and that isn't a failure of the system, it's the system working. Most internal tools solve a narrow, local need; a small number turn out to generalize, and those get pulled forward.

The integration path matters as much as the permission: once a proof-of-concept demonstrates value at showcase, its learnings or modular components are folded back into the core system under the oversight of the responsible PO group — not merged ad hoc by whoever built it.

The Rubber-Stamp Risk The verification system in Section 5 answers the cognitive-overload objection on paper — but human nature under deadline pressure is a different reviewer. If a single accountable human receives ten times the volume of specs, diffs, and release candidates, the temptation to default to trusting the Reviewer agent is real. And if that happens, "the Reviewer agent missed it" becomes the new "the team was accountable" — the exact diffusion of responsibility this model exists to eliminate, reintroduced through the back door. What keeps spot-checking honest at 10× volume —

sampling-based deep dives, enforced friction at high-risk release boundaries, something else entirely — is an open design question, and a genuinely hard one.

The Developer-to-PO Identity Question The model quietly assumes that because an engineer *can* run an agentic team, they want to be accountable for business value. Writing code and owning a value stream exercise different muscles, and many excellent engineers actively dislike stakeholder management and product positioning. Section 7's "honorable path" gestures at this; it does not resolve it. What the role split looks like for those who want to remain purely technical — deep specialists, tooling craftspeople, system architects *within* someone else's agentic team — deserves more thought than this paper gives it.

The Capture Risk Section 3.5 warns against picturing the support functions as human committees — but a warning in a paper does not prevent an organization from doing it anyway. Enabling services have a well-documented tendency to drift into approval gates, especially where middle managers need something to hold. What structural properties keep Curation and Routing genuinely pull-based over time, rather than for the first six months, is an open question — and organizational history gives ample reason for skepticism.

Auditing Accountability Without Producing Fear Section 7 states the principle — blameless review for good-faith decisions, explicit liability limits — but a principle is not a mechanism. How outcome accountability is actually audited, in a way that distinguishes a sound verification process that met an unlucky edge case from a bypassed one that chased speed, is unresolved here. Whoever designs that mechanism well will have solved one of the harder problems in the whole space.

Failure Modes and the Way Back Any organization experimenting with this should define, in advance, what failure looks like and how to reverse: which metrics trigger a pause (delivery quality, PO burnout signals, architectural drift), who has authority to call it, and what the fallback structure is. The five sketches in Section 10 are deliberately reversible in their early phases — an experiment that can't be unwound isn't an experiment, it's a bet.

9. Closing Thought

The point of this paper is not that any organization should adopt this model. The point is that the question it forces — *where does accountability live when execution is no longer scarce?* — is coming regardless of what anyone adopts. The current machinery answers a question nobody is asking anymore; what replaces it is still genuinely open.

If this hypothesis is wrong, it would be useful to know exactly where — that's what it's for.

10. If You Wanted to Test the Idea: Five Sketches

Not adoption paths — ways an organization could *probe the hypothesis* at low stakes, matched to different risk appetites. What all five share: none require expanding the development organization, and all are reversible in their early phases.

One honest qualification. "No new hires" is close to true for delivery capacity, but not a blanket promise. The three load-bearing functions in Section 5 — Curation, Routing, Governance — plus standing legal and compliance access would in practice require re-deploying senior capability, and in some organizations adding it. That is a small investment against the capacity change, but it should be planned for rather than discovered.

Start Small One team, one quarter, one clear before/after number. Pick your most agent-comfortable team, let it run as a single-PO agentic unit, measure the delivery multiple and accountability clarity, then decide whether the hypothesis survived contact with reality. *Tests:* the capacity claim with minimal risk. *Catch:* slow to produce evidence. *Fits:* risk-averse leadership, regulated industries, a first probe anywhere.

Big Bang Full cutover on a set date — all teams dissolve into agentic units simultaneously. *Tests:* the model whole, including the coordination load no pilot reveals. *Catch:* no pilot data if it wobbles, and twenty new POs need mentoring, legal support, and answers to the Open Questions on day one, not gradually. *Fits:* honestly — almost no one; included for completeness and because someone will try it anyway.

Shadow Mode Agentic teams run in parallel with existing teams on the same work items, without replacing anyone. Compare outputs for a sprint or two before any human changes role. *Tests:* the output claim with zero personnel risk. *Catch:* doubles cost temporarily, and "this is just a test" psychology can mask what full commitment would surface. *Fits:* skeptical stakeholders who want evidence before opinion.

Volunteer Wave Open the experiment to whoever wants in first; let visible results attract (or repel) the next cohort. *Tests:* whether the model spreads on merit rather than mandate. *Catch:* volunteers may be your strongest performers — early results might not generalize. *Fits:* organizations where trust in peers exceeds trust in leadership mandates.

Franchise One business unit or product line runs the full experiment — POs, show-case cycle, adoption decisions, the works — while everyone else watches. If it proves out, the *pattern*, not the people, gets replicated. *Tests:* the complete model with contained blast radius. *Catch:* slowest path to any org-wide conclusion, and the franchise unit needs real autonomy or it isn't a fair test. *Fits:* large, multi-unit organizations where one division can credibly serve as a sandbox.

A Note on How This Paper Was Made

This paper was produced by the model it describes: one accountable human Product Owner and an agentic team of several frontier models at the time, handling drafting, critique, and revision. The verification layer worked — several independent agent reviews caught the first versions' weakest claims, and the accountable human overruled the agents more than once, including on the question of what this paper should be.

It is offered as a point of view, freely — a starting provocation for anyone beginning to think about agentic team structures, not a preview of a consulting engagement.

About This Paper

Jesper Karlsson is a UX and product designer, and founder of Otrobonita AI Labs. He has spent three decades in human-computer interaction, enterprise design systems and software development, most recently building autonomous agentic production systems.

Disagreement is the point. If part of this is wrong — and some of it will be — the author would rather hear it than not.

Cite as Karlsson, J. (2026). *The Bottleneck Has Moved: What if every developer became a Product Owner?* Otrobonita AI Labs. otrobonita.com/whitepapers. DOI: 10.5281/zenodo.21622654

Licence Creative Commons Attribution 4.0 (CC BY 4.0). Share it, quote it, argue with it — attribution is all that is asked.

Contact otrobonita.com · otrobonita.com/whitepapers · jesper@otrobonita.com